

Guia Prático de Refatoração e Melhorias

Implementação Passo a Passo para Evolução de Arquitetura, Segurança e UX do Repositório

Este manual técnico instrui, passo a passo, como implementar as melhorias arquiteturais e de segurança sugeridas para o ecossistema `lanchonete-simulator`. As alterações focam no isolamento correto de dados, otimização do tráfego através de payloads JSON estruturados e governança de conexões com o MySQL.

Passo 1: Padronização dos Status na Base de Dados (MySQL)

A tabela original utiliza um tipo genérico `VARCHAR(50)` para armazenar o estado do pedido. Para blindar o sistema contra inconsistências ortográficas (ex: "pendente" minúsculo vs. "Pendente"), vamos migrar para uma restrição do tipo `ENUM`.

Ação Prática:

Execute o seguinte script de migração no console do seu cliente MySQL ou ferramenta de administração:

```
-- Selecciona a base de dados
USE lanchonete;

-- Altera a estrutura do campo status forçando restrição rígida
ALTER TABLE pedidos
MODIFY COLUMN status ENUM('Pendente', 'Preparando', 'Pronto', 'Entregue') NOT NULL DEFAULT
'Pendente';
```

Vantagem Técnica da Modificação

Se o código frontend tentar injetar por acidente uma string inválida ou malformatada, o próprio motor do MySQL rejeitará a transação, mantendo a integridade sem a necessidade de criar regras complexas de validação no código Python.

Passo 2: Otimização de Performance com Connection Pool

Atualmente, o arquivo `database.py` abre e fecha conexões físicas de rede para cada consulta executada. Para mitigar a sobrecarga de processamento (overhead), vamos adotar o mecanismo de pooling nativo do driver MySQL.

Ação Prática:

Abra o arquivo `database.py` e substitua o código existente por esta estrutura otimizada:

```

import mysql.connector
from mysql.connector import Error
from mysql.connector import pooling

class Database:
    # Atributo estático de classe que gerencia o pool de conexões
    _pool = None

    def __init__(self):
        # Inicializa o Pool apenas uma vez durante o ciclo de vida da aplicação
        if Database._pool is None:
            try:
                Database._pool = pooling.MySQLConnectionPool(
                    pool_name="lanchonete_pool",
                    pool_size=5, # Mantém até 5 conexões ativas prontas para reuso
                    host='localhost',
                    user='root',
                    password='Fabio123@',
                    database='lanchonete'
                )
            except Error as e:
                print(f"Erro crítico ao inicializar o Pool de Conexões: {e}")

    def connect(self):
        try:
            # Em vez de criar uma nova conexão, solicita uma já aberta do Pool
            return Database._pool.get_connection()
        except Error as e:
            print(f"Erro ao capturar conexão do Pool: {e}")
            return None

    def close(self):
        # Método mantido para compatibilidade, mas o fechamento agora devolve a conexão ao pool
        pass

    def execute_query(self, query, params=None):
        conn = self.connect()
        if conn:
            cursor = conn.cursor()
            try:
                cursor.execute(query, params)
                conn.commit()
                return cursor
            except Error as e:
                print(f"Erro ao executar a consulta: {e}")
                return None
            finally:
                cursor.close()
                conn.close() # Importante: Libera a conexão de volta ao pool imediatamente
        return None

    def fetch_all(self, query, params=None):
        conn = self.connect()
        if conn:
            cursor = conn.cursor(dictionary=True)
            try:
                cursor.execute(query, params)

```

```

        return cursor.fetchall()
    except Error as e:
        print(f"Erro ao buscar dados: {e}")
        return []
    finally:
        cursor.close()
        conn.close() # Libera a conexão
return []

def fetch_one(self, query, params=None):
    conn = self.connect()
    if conn:
        cursor = conn.cursor(dictionary=True)
        try:
            cursor.execute(query, params)
            return cursor.fetchone()
        except Error as e:
            print(f"Erro ao buscar dado único: {e}")
            return None
    finally:
        cursor.close()
        conn.close() # Libera a conexão
return None

```

Passo 3: Refatoração para API REST nativa com JSON (Backend)

Para alinhar o backend com as chamadas assíncronas do frontend, vamos refatorar a rota de cadastro no arquivo `app.py` para interceptar objetos JSON padronizados, inserindo validações defensivas do tipo de dado recebido.

Ação Prática:

Substitua o método `cadastrar_pedido()` atual no arquivo `app.py` pelo código abaixo:

```

@app.route('/pedidos', methods=['POST'])
def cadastrar_pedido():
    try:
        # Intercepta os dados encapsulados em formato JSON
        data = request.get_json()

        if not data:
            return jsonify({'success': False, 'message': 'Payload JSON não fornecido ou inválido'}), 400

        cliente = data.get('cliente', '').strip()
        lanche = data.get('lanche', '').strip()
        quantidade_raw = data.get('quantidade')
        status = data.get('status', 'Pendente')

        # Validações defensivas estritas antes de persistir no banco
        if not cliente or not lanche:
            return jsonify({'success': False, 'message': 'Campos "cliente" e "lanche" são obrigatórios'}), 400

        try:
            quantidade = int(quantidade_raw)
            if quantidade <= 0:
                raise ValueError()
        except (ValueError, TypeError):
            return jsonify({'success': False, 'message': 'A quantidade deve ser um número inteiro superior a zero'}), 400

        # Executa inserção segura através da camada Model
        if pedido_model.cadastrar(cliente, lanche, quantidade, status):
            return jsonify({'success': True, 'message': 'Pedido cadastrado com sucesso!'}), 201
        else:
            return jsonify({'success': False, 'message': 'Ocorreu um erro interno no banco de dados ao salvar.'}), 500

    except Exception as e:
        # Tratamento de erro ocultando detalhes sensíveis do sistema em produção
        print(f"[LOG ERRO INTERNO]: {str(e)}") # Log visível apenas ao administrador do servidor
        return jsonify({'success': False, 'message': 'Ocorreu um erro inesperado no processamento do pedido.'}), 500

```

Passo 4: Sincronização do Envio JavaScript com o Novo Backend

Para completar o ciclo de melhoria, precisamos ajustar o arquivo JavaScript responsável por enviar o formulário (geralmente mapeado no script `cadastrar.js`) para que envie a requisição formatada como JSON puro de forma assíncrona.

Ação Prática:

Modifique a lógica da função de submissão do formulário no seu arquivo JavaScript do frontend:

```

async function enviarFormularioCadastro(event) {
  event.preventDefault(); // Impede o recarregamento tradicional da página

  const submitBtn = document.querySelector('button[type="submit"]');
  const clienteInput = document.getElementById('cliente').value;
  const lancheInput = document.getElementById('lanche').value;
  const quantidadeInput = document.getElementById('quantidade').value;
  const statusSelect = document.getElementById('status').value;

  // Constrói o objeto literal correspondente ao esperado pela API Flask
  const payload = {
    cliente: clienteInput,
    lanche: lancheInput,
    quantidade: parseInt(quantidadeInput, 10),
    status: statusSelect
  };

  // Ativa travas visuais para UX (Impede cliques múltiplos acidentais)
  submitBtn.disabled = true;
  submitBtn.textContent = 'Processando...';

  try {
    const response = await fetch('/pedidos', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json' // Alerta o Flask sobre o envio de JSON puro
      },
      body: JSON.stringify(payload)
    });

    const resultado = await response.json();

    if (response.ok && resultado.success) {
      showMessage(resultado.message, 'success');
      setTimeout(() => {
        window.location.href = '/pedidos'; // Redirecionamento limpo
      }, 1200);
    } else {
      showMessage(resultado.message || 'Falha ao processar operação', 'error');
      submitBtn.disabled = false;
      submitBtn.textContent = 'Cadastrar Pedido';
    }
  } catch (error) {
    console.error('Erro de conexão:', error);
    showMessage('Não foi possível conectar ao servidor backend.', 'error');
    submitBtn.disabled = false;
    submitBtn.textContent = 'Cadastrar Pedido';
  }
}

```

⚠️ Atenção com Segurança em Produção (app.run)

No bloco final do seu arquivo `app.py`, certifique-se de desativar a flag `debug=True` ao subir a aplicação para um servidor de produção público. O modo debug ativo permite a execução remota de códigos arbitrários através do console interativo exposto no navegador, gerando uma brecha de segurança crítica.